Academic Year: 2026-2027 **Name of Student: Nishant Suryarao**
Semester: III **Student ID:26204001**
Class / Branch: SE IT **Date of Performance:06/10/2026**
Subject: **Date of Submission:06/10/2026**
Name of Instructor:

Experiment No.06 Aim:

Implement N-Queens problem using backtracking.

Code:

```
#include <stdio.h>
#define N 4 int
board[N][N];
// Function to check if placing queen at board[row][col] is
safe int isSafe(int row, int col) {
 int i, j;
 // Check column  for (i = 0; i
< row; i++)  if (board[i][col])
return 0;  // Check upper-
left diagonal
 for (i = row, j = col; i >= 0 && j >= 0; i--, j--)  if
(board[i][j]) return 0;  // Check upper-right
diagonal
 for (i = row, j = col; i >= 0 && j < N; i--, j++)  if
(board[i][j]) return 0;  return 1;
}
// Backtracking function
int solve(int row) {if (row == N) return 1; // All queens placed
for (int col = 0; col < N; col++) {  if (isSafe(row, col)) {
board[row][col] = 1; // Place queen  if (solve(row + 1)) return
1; // Move to next row  board[row][col] = 0; // Backtrack
 } }
return 0;
}
// Print the board void
printBoard() {  for (int i =
0; i < N; i++) {  for (int j =
```

```
0; j < N; j++)  printf("%d ",
board[i][j]);  printf("\n");
 } }
int main() {  if
(solve(0))
printBoard();
else
 printf("No solution\n");
return 0; }
```

Output Screen Shots:

```
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~/Desktop/Nishant$ gcc -o exp6 exp6.c
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~/Desktop/Nishant$ ./exp6
0 1 0 0
0 0 0 1
1 0 0 0
0 0 1 0
apsit@apsit-HP-Pro-Tower-280-G9-E-PCI-Desktop-PC:~/Desktop/Nishant$
```

Conclusion:

The N-Queens problem was successfully implemented using the backtracking technique. The algorithm efficiently places queens while checking row, column, and diagonal constraints, demonstrating the use of recursion and backtracking to find a valid solution.

Experiment 6: N-Queens Using Backtracking

Definition: The N-Queens problem involves placing N queens on an \(N \times N\) chessboard so that no two queens attack each other. No two queens can share the same row, column, or diagonal.

Algorithm:

1. Start with an empty chessboard.
2. Begin with the first row.
3. Try placing a queen in each column of the current row.
4. Check whether the position is safe.
5. If safe, place the queen and move to the next row.
6. If no safe position is available, backtrack and move the previous queen.
7. Repeat until all N queens are placed.
8. Display the solution board.
9. Stop.

Conclusion: Thus, the N-Queens problem was successfully solved using backtracking by checking row, column, and diagonal constraints.

Complexity: Worst-case time is approximately \(O(N!)\) for the standard backtracking approach.